

Linked Servers on MS SQL Part 3

By Don Schlichting

Introduction

In Parts 1 and 2, we introduced the Linked Server. Starting with what a Linked Server is, the benefits to using them, as well as when to avoid them. The Distributed Transaction Coordinator, DTC, was touched on. In addition, our first examples were to create links to an Excel spreadsheet, during which, four-part naming was reviewed. The functions `sp_addlinkedserver`, `openrowset`, and `sp_dropserver` were used. Moreover, the differences between Direct Referencing and using the `openrowset` were explained. The final examples were to link an Access database, and a SQL database using Windows security. In this third part, we will continue with linked SQL server security.

Windows Security

Let's create a Linked SQL Server with standard Windows security as a starting point. Open Query Analyzer and connect to your local SQL with Windows authentication.



From inside Query Analyzer, we will use the `sp_AddLinkedServer` function to create our link:

```
EXEC sp_addlinkedserver
    @server = 'TEST1',
    @srvproduct = 'SQLServer OLEDB Provider',
    @provider = 'SQLOLEDB',
    @datasrc = 'InfoNet'
```

@server is the Link name. @datasrc is the remote SQL server.

As a test, run a simple select out of our linked server:

```
SELECT *
FROM TEST1.Northwind.dbo.Orders
```

The full orders table should be returned. If not, verify you have adequate permissions on the remote machine. Then check for type errors in `addlinkedserver` statement.

Alternatively, to create the link from inside Enterprise Manager, navigate your local server tree to Security, Linked Servers, right click, and select "New Linked Server." Select SQL Server as the server type, and use the Linked Server box for the remote server name. In this case, the remote server name will also be the Linked Server name.

The connection just made will pass your Windows login credentials to the remote server. If the Windows account does not have SQL permissions on the remote, all queries will fail. Open the security properties for the link we just created. From Enterprise Manager> Security> Linked Servers, right click on the TEST1 link. The center maps local logins to others on the remote. These must be explicitly created. Any logins not found here, or if it is empty, as in our case, then the radio buttons on the bottom take over.



sp_addLinkedSrvLogin

To create the same security context from Query Analyzer, use the sp_addLinkedSrvLogin function. This function handles the same basic options as Enterprise Manager. The syntax is straightforward:

```
sp_addlinkedsevrlogin [ @rmtsrvname = ] 'rmtsrvname'
    [, [ @useself = ] 'useself' ]
    [, [ @locallogin = ] 'locallogin' ]
    [, [ @rmtuser = ] 'rmtuser' ]
    [, [ @rmtpassword = ] 'rmtpassword' ]
```

@rmtsrvname - is the linked server name previously created.

@useself - if set to true, Windows authentication is used. False will use SQL security.

@locallogin - Local SQL user ID.

@rmtuser - Remote SQL user ID.

@rmtpassword - Remote SQL password.

For the above example, where Windows security was used exclusively, the syntax is:

```
EXEC sp_addlinkedsevrlogin 'TEST1', 'true'
```

Not be made

If the "Not be made" radio button is selected, then any logins not explicitly mapped will be denied. In our case, with no logins defined, all queries will fail. The test sql statement:

```
SELECT *  
FROM TEST1.Northwind.dbo.Orders
```

Will fail with error:

Server: Msg 7416, Level 16, State 1, Line 1

Access to the remote server is denied because no login-mapping exists.

Be made without using a security context

When the "Be made without using a security context" is selected, any queries submitted by a login not defined will be passed to the remote server using guest permissions.

From Query Analyzer, the security syntax is:

```
EXEC sp_addlinkedsevrlogin 'TEST1', 'false', NULL, NULL, NULL
```

Be made using the login's current security context

This has been the only permission level used so far. Queries submitted by a login not explicitly mapped are passed with the current Windows login security. The Windows account must have permissions on both the local and remote SQL databases.

From Query Analyzer, the security syntax is:

```
EXEC sp_addlinkedsevrlogin 'TEST1', 'true'
```

Be made using this security context

Selecting this security context maps any logins not explicitly defined to one specific SQL account on the remote. The password entered here will not automatically synchronize password changes with the remote. Any password changes made on the remote must be manually changed here as well.

From Query Analyzer, the security syntax is:

```
EXEC sp_addlinkedsevrlogin 'TEST1', 'false', 'userLocal', 'userRemote', 'remotePassword'
```

To specify a local Windows login, rather than a SQL login, use the keyword word "domain\".

```
EXEC sp_addlinkedsevrlogin 'TEST1', 'false', 'domain\localWinUser', 'userRemote',  
'remotePassword'
```

Linked Servers on MS SQL Part 3 - Page 2

By Don Schlichting

Impersonate

By using the Local Login dialog boxes, we can map local SQL users to remote users. The Impersonate checkbox maps the local user, (sa in this example), to the remote login "sa". The local login must match exactly with the remote login for this to work.



Remote User

One to one user mappings can be done by selecting a Local Login, leaving Impersonate unchecked and specifying a Remote User and Password.

From Query Analyzer, the security syntax is:

```
EXEC sp_addlinkedsevrlogin 'TEST1', 'false', 'donLocal', 'donRemote', 'remotePassword'
```

Sp_DropLinkedSrvLogin

To remove a linked server login, use the sp_droplinkedsevrlogin function. This function has only two arguments, the name of the server, and the login to be removed. The syntax for removing the login "donLocal" from above is:

```
EXEC sp_droplinkedsevrlogin 'TEST1', 'donLocal'
```

When the linked server is first created, a default login mapping is created. To remove this mapping, run the same statement using NULL as the user name.

```
EXEC sp_droplinkedsevrlogin 'TEST1', NULL
```

PERFORMANCE

Distributed queries will be slower than single server selects. The fact that we are communicating over the wire to remote machines implies a performance hit. However, there are methods to minimize performance problems.

- Use the OpenQuery function rather than a direct reference. (See part 2 for OpenQuery examples.)
- Execute stored procedures on the remote if possible. To run a remote stored procedure, use the syntax:

```
SELECT * FROM OpenQuery(TEST1, 'northwind.dbo.[Ten Most Expensive Products]')
```

- Avoid data conversion operations if possible.
- Set Collation to true when both servers have the same sort order and character set. This will prevent the remote server from sending the entire table over the wire to the local server when a WHERE clause is used. The collation is an option of the sp_ServerOption function. For our example, the syntax would be:

```
EXEC sp_serveroption 'TEST1', 'collation compatible', 'true'
```

Conclusion

As with all things programmed, the necessary security will add additional complexity. However, SQL has given us some very usable tools to deal with it.

Hey, is your BOL a little sparse on SQL to Oracle linked servers. Join us next month for Linked Servers Part 4: Oracle.